ACT->futex系统调用

1. ACT模型内

/Users/vel/Desktop/RobotOS/Lerobot/lerobot/src/lerobot/policies/act/configuration_act.py

在 ACT 中，主要的计算开销和线程同步源于两个地方：

```
cam_features = self.backbone(img)["feature_map"]

self.encoder = ACTEncoder(config)
self.decoder = ACTDecoder(config)
```

2. torchvision

`vision/torchvision/models/resnet.py` 中 `ResNet.forward(x)` 跳转到 `_forward_impl(x)` 。

```
    def _forward_impl(self, x: Tensor) -> Tensor:
        # See note [TorchScript super()]
        x = self.conv1(x)   # 第一个卷积层
        x = self.bn1(x)
        x = self.relu(x)
        x = self.maxpool(x)

        x = self.layer1(x)  # 进入 BasicBlock 组成的层
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)

        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.fc(x)

        return x
```

进入 BasicBlock 与 Conv2d , 在 BasicBlock 源码中：

```python
def forward(self, x: Tensor) -> Tensor:
    identity = x
    out = self.conv1(x)  # <--- 这里的 self.conv1 是在 __init__ 里通过 conv3x3() 创建的
    # ...

def conv3x3(in_planes: int, out_planes: int, stride: int = 1, groups: int = 1, dilation
    """3x3 convolution with padding"""
    return nn.Conv2d(   # conv3x3 返回的是一个普通的 nn.Conv2d 实例。
        in_planes,
        out_planes,
        kernel_size=3,
        stride=stride,
        padding=dilation,
        groups=groups,
        bias=False,
        dilation=dilation,
    )
```

此时跳转到 Module._call_impl。

3. torch.nn.modules.conv

nn.Conv2d 的源码位于： torch/nn/modules/conv.py

```python
def _conv_forward(self, input: Tensor, weight: Tensor, bias: Tensor | None):
    if self.padding_mode != "zeros":
        return F.conv2d(
            F.pad(
                input, self._reversed_padding_repeated_twice, mode=self.padding_mode
            ),
            weight,
            bias,
            self.stride,
            _pair(0),
            self.dilation,
            self.groups,
        )

    return F.conv2d(
        input, weight, bias, self.stride, self.padding, self.dilation, self.groups
    )   # <--- 跳转到 F.conv2d

def forward(self, input: Tensor) -> Tensor:
    return self._conv_forward(input, self.weight, self.bias)
```

/opt/anaconda3/envs/lerobot/lib/python3.10/site-packages/torch/_C/_VariableFunctions.pyi 中

```python
@overload
def conv2d(input: Tensor, weight: Tensor, bias: Optional[Tensor] = None, stride: Union[
@overload
def conv2d(input: Tensor, weight: Tensor, bias: Optional[Tensor] = None, stride: Union[
```

pytorch/aten/src/ATen/native/native_functions.yaml 中

```yaml
- func: conv2d(Tensor input, Tensor weight, Tensor? bias=None, SymInt[2] stride=1, SymI
  dispatch:
    CompositeImplicitAutograd: conv2d_symint  # <--- 注意这里!
```

当执行 F.conv2d 时，调用的链路如下：

1. Python 层：调用 torch.conv2d。
2. 存根文件 (.pyi)：引导解释器去查找 torch._C（即 libtorch_python.so）。
3. C++ 桥接层 (python_variable_methods.cpp)： PyTorch 的自动生成工具根据你这段 YAML，在 C++ 侧生成了一个名为 THPVariable_conv2d 的包装函数。它负责把 Python 的 stride=[1,1] 转换成

C++ 的 SymIntArrayRef。把 Python 的 weight 转换成 at::Tensor。

4. 跳转到 ATen：包装函数最后会执行：

```
return wrap(at::conv2d_symint(input, weight, bias, stride, padding, dilation, groups));
```

4. torchC

pytorch/aten/src/ATen/native/Convolution.cpp

```
at::Tensor conv2d_symint(
    const Tensor& input_, const Tensor& weight, const std::optional<Tensor>& bias_opt,
    SymIntArrayRef stride, SymIntArrayRef padding, SymIntArrayRef dilation, c10::SymInt
  // See [Note: hacky wrapper removal for optional tensor]
  c10::MaybeOwned<Tensor> bias_maybe_owned = at::borrow_from_optional_tensor(bias_opt);
  const Tensor& bias = *bias_maybe_owned;

  TORCH_CHECK(
    !bias.defined() || bias.dtype() == input_.dtype(),
    "Input type (",
    input_.dtype().name(),
    ") and bias type (",
    bias.dtype().name(),
    ") should be the same");

  auto [input, is_batched] = batchify(input_, /*num_spatial_dims=*/ 2, "conv2d");
  Tensor output;
  if (at::isComplexType(input_.scalar_type())) {
    output = complex_convolution(input, weight, bias, stride, padding, dilation, false,
  } else {
    output = at::convolution_symint(input, weight, bias, stride, padding, dilation, fal
  }     // 这里调用了 ATen Dispatcher。由于在树莓派 CPU 上运行，它会跳过 CUDA 路径，选择 CPU 后端
  return is_batched ? std::move(output) : output.squeeze(0);
}
```

pytorch/aten/src/ATen/Parallel.h

并行计算中，系统会爆发大量futex耗时

```
template <class F>
inline void parallel_for(
    const int64_t begin,
    const int64_t end,
    const int64_t grain_size,
    const F& f);


#if AT_PARALLEL_OPENMP  // 在树莓派上，PyTorch 通常默认使用 OpenMP 路径
#include <ATen/ParallelOpenMP.h> // IWYU pragma: keep
#elif AT_PARALLEL_NATIVE
#include <ATen/ParallelNative.h> // IWYU pragma: keep
#endif
```

pytorch/aten/src/ATen/ParallelOpenMP.h 中，线程分叉

```
#ifdef _OPENMP
namespace at::internal {
template <typename F>
inline void invoke_parallel(
    int64_t begin,
    int64_t end,
    int64_t grain_size,
    const F& f) {
  std::atomic_flag err_flag = ATOMIC_FLAG_INIT;
  std::exception_ptr eptr;

#pragma omp parallel
  {
    // choose number of tasks based on grain size and number of threads
    // can't use num_threads clause due to bugs in GOMP's thread pool (See
    // #32008)
    int64_t num_threads = omp_get_num_threads();
    if (grain_size > 0) {
      num_threads = std::min(num_threads, divup((end - begin), grain_size));
    }

    int64_t tid = omp_get_thread_num();
    int64_t chunk_size = divup((end - begin), num_threads);
    int64_t begin_tid = begin + tid * chunk_size;
    if (begin_tid < end) {
      try {
        internal::ThreadIdGuard tid_guard(tid);
        f(begin_tid, std::min(end, chunk_size + begin_tid));    // f 就是卷积运算。
      } catch (...) {
        if (!err_flag.test_and_set()) {
          eptr = std::current_exception();
        }
      }
    }
  }
  if (eptr) {
    std::rethrow_exception(eptr);
  }
}
} // namespace at::internal
#endif // _OPENMP
```

`#pragma omp parallel` 这是 OpenMP 的指令。主线程（执行 Python 推理的那个）会在这里强制停下，并发出指令唤醒 OpenMP 线程池里的其他核心。

这个下面的的右括号有一个隐式同步屏障 (Implicit Barrier)，主线程必须在这里死等，直到 begin_tid < end 逻辑下的所有核心全部执行完 f(...)。